

Sockets in System Programming

A practical reference guide for POSIX socket programming

1. socket() — Creating a Socket

The `socket()` call is the starting point for all socket programming:

```
int socket(int domain, int type, int protocol);
```

Parameter	Common Values
domain	AF_INET (IPv4), AF_INET6 (IPv6), AF_UNIX (local IPC)
type	SOCK_STREAM (TCP), SOCK_DGRAM (UDP)
protocol	Usually 0 (auto-select)

2. Key System Calls

Server Side

Call	Purpose
socket()	Create the socket
bind()	Attach socket to an IP + port
listen()	Mark socket as passive (waiting for connections)
accept()	Block and wait for a client connection

Client Side

Call	Purpose
socket()	Create the socket
connect()	Initiate connection to server

Both Sides

Call	Purpose
send() / recv()	Send and receive data
close()	Close the socket

3. TCP vs UDP

Feature	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery	No guarantee
Order	In-order	Not guaranteed
Speed	Slower	Faster
Use case	HTTP, FTP, SSH	DNS, video streaming

4. Important Structs

The `sockaddr_in` struct is used for IPv4 addressing:

```
struct sockaddr_in {
    sa_family_t    sin_family;    // AF_INET
    in_port_t      sin_port;      // port (network byte order)
    struct in_addr sin_addr;      // IP address
};
```

Always use **htons()** for ports and **inet_pton()** for IP addresses to ensure correct byte ordering.

5. Byte Order

Different machines store bytes differently. Sockets require **network byte order (big-endian)**:

Function	Purpose
htons()	Host → Network (short — for ports)
htonl()	Host → Network (long — for addresses)
ntohs()	Network → Host (short)
ntohl()	Network → Host (long)

6. Error Handling

Unlike pthreads, most socket calls **return -1 and set errno** on failure:

```
if (bind(sockfd, ...) == -1) {
    perror("bind failed");
}
```

7. Connection Flow Summary

TCP Server

```
socket() → bind() → listen() → accept() → send()/recv() → close()
```

TCP Client

```
socket() → connect() → send()/recv() → close()
```